

Investigating positional accuracy by double integrating accelerometer data

September 4, 2026

Contents

1	Introduction	2
2	Numerical Integration and code application	3
2.1	Testing the waters	3
2.2	Axis definitions for phone directions	5

1 Introduction

Inertial Odometry has been studied with a variety of applications, whenever you need to know the precise location of something instantly, chances are that double integrating accelerometer data outpaces GPS solutions. From robotics to step trackers, they all rely on the property that the area under a accelerometer graph can provide enough data used to compute the displacement of the object. In this investigation we will explore the accuracy of double integrating accelerometer data recorded on a phone during travel to find path traveled to get to school and create a visual representation of the position with respect to time and comparing with GPS data.

2 Numerical Integration and code application

Recall that in kinematics integrating acceleration gives velocity, and integrating once again gives displacement. For discrete-time data, this cannot be solve analytically. Instead we refer to the trapezoidal method, illustrated below:

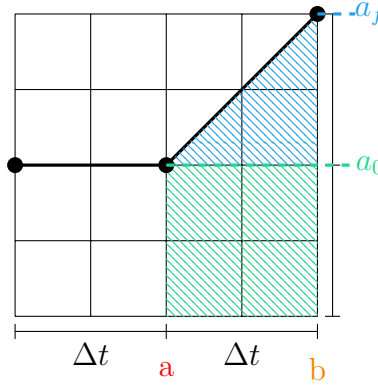


Figure 1: Integrating two data points

To calculate the area bounded by the x axis and the line created from two datapoints, we use basic geometry equations

$$\Delta v = \int_a^b a(t) dt = \Delta t \cdot a_0 + \frac{1}{2} \Delta t \cdot a_f \quad (1)$$

which is thus computed in python code

```
dt = final_time - initial_time
dx_accel = final_accel - initial_accel
dx_vel = (dt * initial_accel) + ((1/2) * dt * dx_accel)
# triangle area + square area = area under accel graph = change in velocity
```

2.1 Testing the waters

the same idea will be repeated for each axis and afterwards applied to the calculated velocity values to find displacement. As a basic test of the practicality I moved my phone left and right 5 times on my table starting and ending at 0 velocity. Below we get the following graph of raw data of x y and z acceleration with respect to time:

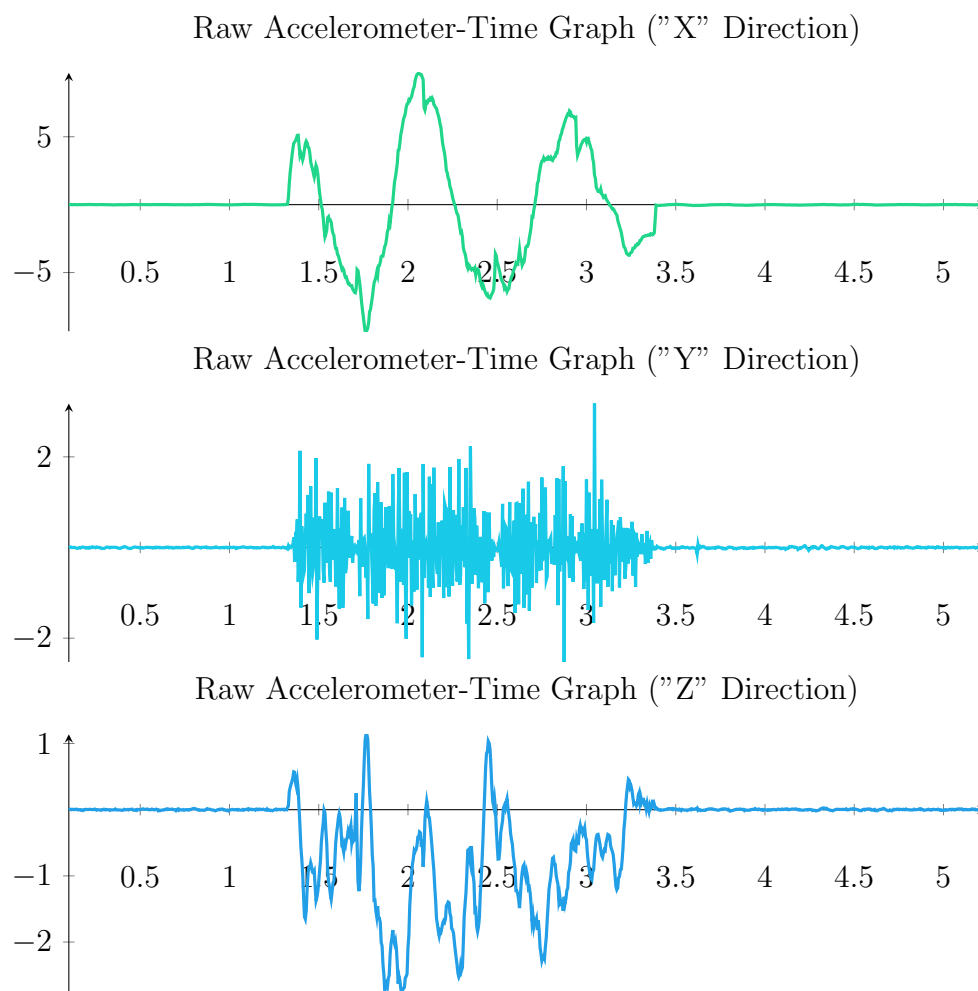


Figure 2: Raw Accelerometer Data On X, Y and Z axis Respectively

2.2 Axis definitions for phone directions

It appears that the X axis best represents left-right movement, after further experimentation, the axis of the phone is established as illustrated:

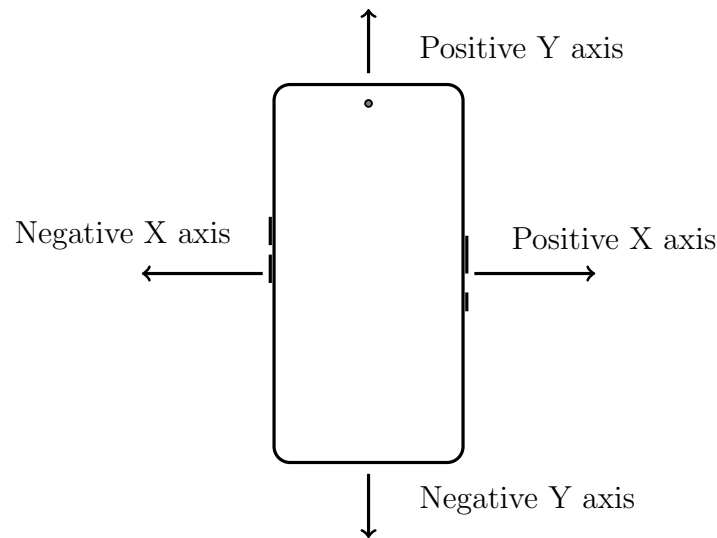


Figure 3: Diagram of respective axis to the phone

And integrating the x-axis with the aforementioned algorithm we get this series of integrations

Clearly this looks...like it could be better. One very apparent issue is that the integrated velocity-time graph ends with a velocity lower than zero! The reason for why it happened could be because the accelerometer data is not centered, but looking at it again it is centered as it starts with 0 acceleration and ends with 0 acceleration. To center it we use the following code:

```
average_acceleration = sum(acceleration)/len(acceleration) # The sum of the accel
processed = [] # A blank array
for datum in acceleration:
    processed.append(datum-average acceleration) # Subtracting the bias from each
```

and we get the following integrations

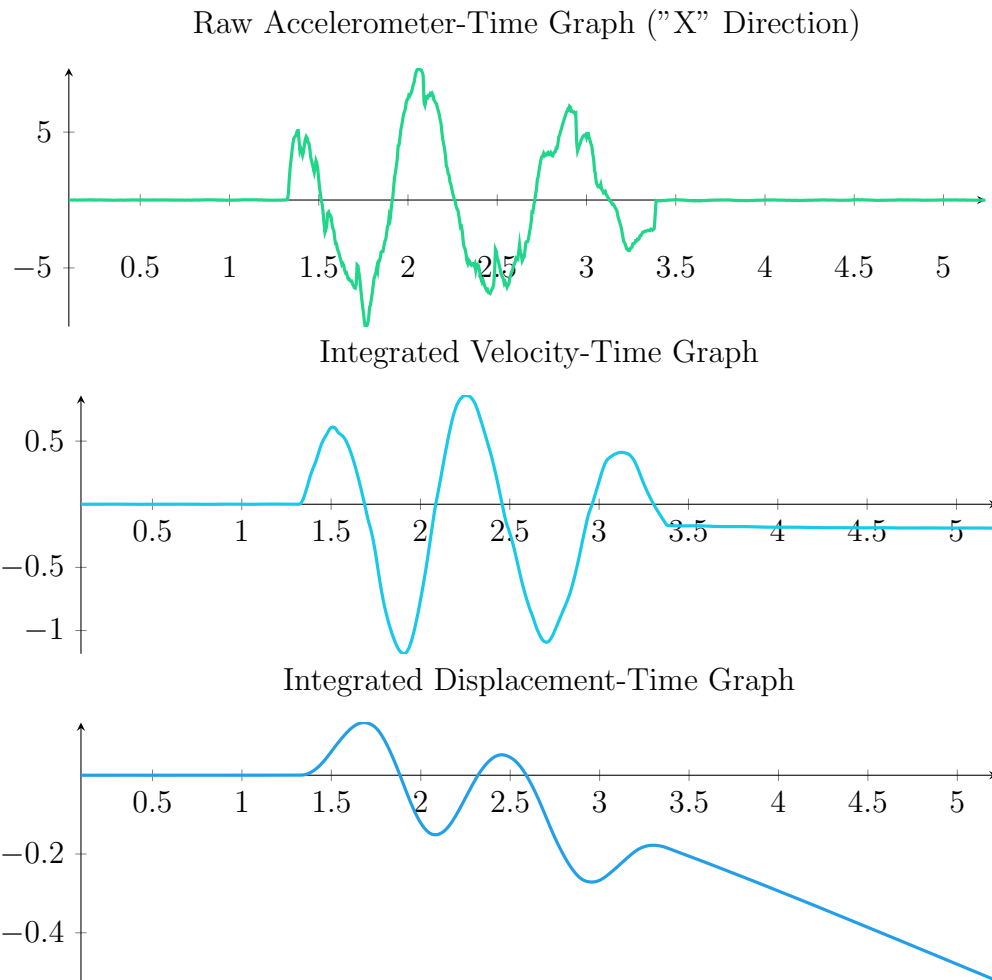


Figure 4: The Progressive Progress of Numerically integrating accelerometer data